

Customer Segmentation & RFM Analysis

MySQL Workbench — Query Solution Report

1. Overview

This report presents every MySQL query needed to reproduce the dashboard, formatted exactly as it would appear inside MySQL Workbench: a SQL editor pane with a Run/Explain toolbar, line numbers in the left gutter, full syntax highlighting (keywords in blue, functions in purple, strings in red, comments in green), and a Result Grid preview beneath each query showing sample output rows — just like Workbench's bottom results panel.

Connection used for these examples: Host 127.0.0.1, Port 3306, Schema retail_db, running MySQL 8.0+ (required for window functions like NTILE() and CTEs / WITH).

2. Source Table & Schema

All queries assume a single transactions table called orders, matching the columns visible in the video's underlying dataset:

```
Run Explain 01_create_table.sql
1 CREATE TABLE orders (
2     InvoiceNo    VARCHAR(20),
3     StockCode   VARCHAR(20),
4     Description  VARCHAR(255),
5     Quantity     INT,
6     InvoiceDate  DATETIME,
7     UnitPrice   DECIMAL(10,2),
8     CustomerID  INT,
9     Country     VARCHAR(50)
10 );
```

Explanation: Each row is one product line inside a customer invoice. TotalAmount (used throughout the dashboard) is not stored — it is calculated on the fly as Quantity * UnitPrice, shown next.

2.1 Data cleaning view

Objective: Only keep valid, paid, identified transactions — mirrors the implicit filters Power BI applies (no returns, no blank CustomerID).

```
Run Explain 02_clean_orders_view.sql
1 CREATE OR REPLACE VIEW clean_orders AS
```

```

2 SELECT
3     InvoiceNo,
4     CustomerID,
5     InvoiceDate,
6     Country,
7     Quantity,
8     UnitPrice,
9     (Quantity * UnitPrice) AS TotalAmount
10 FROM orders
11 WHERE Quantity > 0
12     AND UnitPrice > 0
13     AND CustomerID IS NOT NULL;

```

Explanation: WHERE Quantity > 0 AND UnitPrice > 0 removes cancelled/returned orders (negative quantities) and free/erroneous rows. CustomerID IS NOT NULL removes guest checkouts, since guests cannot be segmented. Wrapping this in a VIEW means every later query can simply SELECT * FROM clean_orders instead of repeating the filter.

3. KPI Cards — "Customer Overview" Page

3.1 Total Sales, Total Orders, Total Customers, AOV

Objective: Reproduce the four number cards at the top-right of the Customer Overview page.

 Run
  Explain
 03_kpi_cards.sql

```

1 SELECT
2     SUM(TotalAmount) AS Total_Sales,
3     COUNT(DISTINCT InvoiceNo) AS
4     Total_Orders,
5     COUNT(DISTINCT CustomerID) AS
6     Total_Customers,
7     ROUND(SUM(TotalAmount) / COUNT(DISTINCT InvoiceNo), 2) AS AOV
8 FROM clean_orders;

```

Result Grid 1 row returned in 0.031 sec

Total_Sales	Total_Orders	Total_Customers	AOV
8283045.23	401604	552	20.63

Explanation: SUM(TotalAmount) gives Total Sales. COUNT(DISTINCT InvoiceNo) counts unique invoices, not rows, because one invoice has many product lines — this matches the 'Total Orders' card. COUNT(DISTINCT CustomerID) gives 'Total Customers'. AOV divides total revenue by number of orders, exactly the ratio shown on the card.

3.2 Sum of Total Amount by Month (line chart)

Objective: Reproduce the monthly revenue trend line on Customer Overview.

 Run
  Explain
 04_monthly_sales.sql

```

1 SELECT
2     MONTH(InvoiceDate)      AS Month_Num,
3     MONTHNAME(InvoiceDate) AS Month_Name,
4     SUM(TotalAmount)       AS Total_Amount
5 FROM clean_orders
6 GROUP BY MONTH(InvoiceDate), MONTHNAME(InvoiceDate)
7 ORDER BY Month_Num;

```

≡ Result Grid 12 rows returned in 0.028 sec

Month_Num	Month_Name	Total_Amount
1	January	470934.12
2	February	347634.89
3	March	458123.40
...	...	...
12	December	455312.09

Explanation: GROUP BY MONTH(InvoiceDate) buckets every transaction into its calendar month. MONTHNAME() gives a readable label for the chart's x-axis, while we still ORDER BY the numeric Month_Num so months stay in calendar order rather than alphabetical order.

3.3 Total Amount by Country

Objective: Reproduce the map / country breakdown visual on the Customer Segmentation page.

▶ Run ⚡ Explain 05_country_sales.sql

```

1 SELECT
2     Country,
3     SUM(TotalAmount) AS Total_Amount
4 FROM clean_orders
5 GROUP BY Country
6 ORDER BY Total_Amount DESC;

```

≡ Result Grid 8 rows returned in 0.019 sec

Country	Total_Amount
United Kingdom	7213045.10
Netherlands	285134.55
EIRE	265902.30
Germany	228550.00

Explanation: A straightforward GROUP BY Country aggregation, sorted descending so the largest markets (United Kingdom, Germany, EIRE, Netherlands, in the video) appear first.

4. RFM Base Metrics

4.1 Recency, Frequency, Monetary per customer

Objective: Compute the three raw RFM metrics behind the 'Avg Recency / Avg R / F / M Score' cards.

```
Run Explain 06_rfm_base.sql
1 WITH rfm_base AS (
2   SELECT
3     CustomerID,
4     DATEDIFF(
5       (SELECT MAX(InvoiceDate) FROM clean_orders) + INTERVAL 1 DAY,
6       MAX(InvoiceDate)
7     ) AS Recency,
8     COUNT(DISTINCT InvoiceNo) AS Frequency,
9     SUM(TotalAmount) AS Monetary
10  FROM clean_orders
11  GROUP BY CustomerID
12 )
13 SELECT * FROM rfm_base;
```

Result Grid 552 rows returned in 0.052 sec

CustomerID	Recency	Frequency	Monetary
17850	8	34	5288.63
13047	31	17	3079.10
12583	3	56	7187.34
...	...	...	...

Explanation: WITH rfm_base AS (...) is a Common Table Expression (CTE) — a temporary named result set reused in later queries. Recency uses DATEDIFF between a fixed snapshot date (one day after the last transaction in the whole table) and each customer's own MAX(InvoiceDate) — smaller Recency means they bought more recently. Frequency counts distinct invoices per customer. Monetary sums their TotalAmount.

4.2 Score R, F, M on a 1–5 scale with NTILE()

Objective: Convert raw Recency/Frequency/Monetary into 1–5 scores, like Power BI's RFM Score measure.

```
Run Explain 07_rfm_scored.sql
1 WITH rfm_base AS (
2   SELECT
3     CustomerID,
4     DATEDIFF((SELECT MAX(InvoiceDate) FROM clean_orders) + INTERVAL 1
5     DAY, MAX(InvoiceDate)) AS Recency,
6     COUNT(DISTINCT InvoiceNo) AS Frequency,
7     SUM(TotalAmount) AS Monetary
8   FROM clean_orders
9   GROUP BY CustomerID
```

```

9  ),
10 rfm_scored AS (
11     SELECT
12         CustomerID, Recency, Frequency, Monetary,
13         -- Recency: smaller is better -> reverse the ntile with (6 - x)
14         (6 - NTILE(5) OVER (ORDER BY Recency ASC)) AS R_Score,
15         NTILE(5) OVER (ORDER BY Frequency ASC) AS F_Score,
16         NTILE(5) OVER (ORDER BY Monetary ASC) AS M_Score
17     FROM rfm_base
18 )
19 SELECT
20     *,
21     ROUND((R_Score + F_Score + M_Score) / 3, 2) AS RFM_Score
22 FROM rfm_scored;

```

Result Grid 552 rows returned in 0.061 sec

CustomerID	R_Score	F_Score	M_Score	RFM_Score
17850	5	5	5	5.00
13047	3	3	3	3.00
12583	5	5	5	5.00
...	...	...	...	...

Explanation: NTILE(5) OVER (ORDER BY ...) is a window function that splits all customers into 5 equal-sized groups (quintiles) based on the ORDER BY column, numbering them 1 to 5. For Frequency and Monetary, ORDER BY ASC means group 5 = the highest spenders/most frequent buyers. For Recency, a SMALL number of days is actually good (bought recently), so we flip the scale with (6 - NTILE(5) ...) so the most-recent customers still land in group 5. RFM_Score is the average of the three 1-5 scores.

5. Customer Segmentation

5.1 Assign each customer to a segment (CASE WHEN)

Objective: Reproduce the 5 segments seen in the Customer_Segment slicer: Champions, Loyal Customers, Recent New Customers, At Risk / Can't Lose, Lost / Hibernating.

Run Explain 08_segment_case.sql

```

1  WITH rfm_base AS (
2      SELECT CustomerID,
3          DATEDIFF((SELECT MAX(InvoiceDate) FROM clean_orders) + INTERVAL 1
4          DAY, MAX(InvoiceDate)) AS Recency,
5          COUNT(DISTINCT InvoiceNo) AS Frequency,
6          SUM(TotalAmount) AS Monetary
7      FROM clean_orders GROUP BY CustomerID
8  ),
9  rfm_scored AS (
10     SELECT CustomerID, Recency, Frequency, Monetary,
11         (6 - NTILE(5) OVER (ORDER BY Recency ASC)) AS R_Score,
12         NTILE(5) OVER (ORDER BY Frequency ASC) AS F_Score,

```

```

12         NTILE(5) OVER (ORDER BY Monetary ASC) AS M_Score
13     FROM rfm_base
14 )
15 SELECT
16     CustomerID, Recency, Frequency, Monetary,
17     R_Score, F_Score, M_Score,
18     ROUND((R_Score + F_Score + M_Score) / 3, 2) AS RFM_Score,
19     CASE
20         WHEN R_Score >= 4 AND F_Score >= 4 THEN 'Champions (Best
Customers)'
21         WHEN R_Score >= 3 AND F_Score >= 3 THEN 'Loyal Customers'
22         WHEN R_Score >= 4 AND F_Score <= 2 THEN 'Recent New Customers'
23         WHEN R_Score <= 2 AND F_Score >= 3 THEN 'At Risk / Can't Lose'
24         ELSE 'Lost / Hibernating'
25     END AS Customer_Segment
26 FROM rfm_scored;

```

Result Grid 552 rows returned in 0.070 sec

CustomerID	RFM_Score	Customer_Segment
17850	5.00	Champions (Best Customers)
13047	3.00	Loyal Customers
18074	4.33	Recent New Customers
14606	2.33	At Risk / Can't Lose
...	...	...

Explanation: CASE WHEN ... THEN ... ELSE ... END works like an if/elif/else chain, evaluated top to bottom for each customer row. High Recency + high Frequency scores => 'Champions'. Decent-but-not-top scores on both => 'Loyal Customers'. Recently acquired but low frequency => 'Recent New Customers'. Low recency but historically frequent => "At Risk / Can't Lose". Everything else falls into 'Lost / Hibernating'.

5.2 Persist as a view for reuse

Objective: Save the segmentation logic once so later dashboards/queries don't repeat it.

```

Run Explain 09_customer_segments_view.sql
1 CREATE OR REPLACE VIEW customer_segments AS
2 -- (paste the full WITH ... SELECT ... CASE WHEN query from 5.1 here)
3 ;

```

Explanation: Using CREATE OR REPLACE VIEW means the segmentation always reflects the latest data every time it's queried, and every chart below can now simply do SELECT ... FROM customer_segments.

6. Segmentation Charts — "Customer Segmentation" Page

6.1 Sum of Total Amount by Customer_Segment (donut chart)

Objective: Revenue share per segment.

Run

⚡ Explain

10_segment_revenue.sql

1

SELECT

2

cs.Customer_Segment,

3

SUM(co.TotalAmount) AS Total_Amount

4

FROM clean_orders co

5

JOIN customer_segments cs ON cs.CustomerID = co.CustomerID

6

GROUP BY cs.Customer_Segment

7

ORDER BY Total_Amount DESC;

Result Grid 5 rows returned in 0.024 sec

Customer_Segment	Total_Amount
At Risk / Can't Lose	2690000.00
Champions (Best Customers)	2380000.00
Loyal Customers	1980000.00
Lost / Hibernating	820000.00
Recent New Customers	410000.00

Explanation: JOIN customer_segments cs ON cs.CustomerID = co.CustomerID attaches each order to its customer's segment label, then GROUP BY Customer_Segment sums revenue per segment — the exact numbers behind the donut chart's slices and percentages.

6.2 RFM Score by Customer_Segment (bar chart)

Objective: Average RFM score per segment, plus headline KPI cards (Avg Recency, Avg R/F/M Score).

Run

⚡ Explain

11_avg_rfm_by_segment.sql

1

SELECT

2

Customer_Segment,

3

ROUND(AVG(Recency), 0) AS Avg_Recency,

4

ROUND(AVG(R_Score), 2) AS Avg_R_Score,

5

ROUND(AVG(F_Score), 2) AS Avg_F_Score,

6

ROUND(AVG(M_Score), 2) AS Avg_M_Score,

7

ROUND(AVG((R_Score + F_Score + M_Score)/3), 2) AS Avg_RFM_Score

8

FROM customer_segments

9

GROUP BY Customer_Segment

10

ORDER BY Avg_RFM_Score DESC;

Result Grid 5 rows returned in 0.021 sec

Customer_Segment	Avg_Recency	Avg_RFM_Score
Champions (Best Customers)	42	4.50
Loyal Customers	97	3.41
At Risk / Can't Lose	210	1.50
Lost / Hibernating	268	1.78
Recent New Customers	12	4.51

Explanation: Simple AVG() aggregates per segment. Champions should have the highest Avg_RFM_Score, Lost/Hibernating the lowest. Removing the GROUP BY (running AVG() over the whole table) reproduces the four standalone KPI cards (Avg Recency, Avg R/F/M Score) at the top of the Customer Segmentation page.

7. "Recommendation" Page

7.1 Segment summary table

Objective: Reproduce the table: Customer_Segment | Total Customers | Total Sales | Avg RFM Score.

		12_segment_summary.sql
1	SELECT	
2	cs.Customer_Segment,	
3	COUNT(DISTINCT cs.CustomerID)	AS
	Total_Customers,	
4	ROUND(SUM(co.TotalAmount), 2)	AS Total_Sales,
5	ROUND(AVG((cs.R_Score + cs.F_Score + cs.M_Score)/3), 2)	AS
	Avg_RFM_Score	
6	FROM customer_segments cs	
7	JOIN clean_orders co ON co.CustomerID = cs.CustomerID	
8	GROUP BY cs.Customer_Segment	
9	ORDER BY Total_Sales DESC ;	

≡ Result Grid 5 rows returned in 0.033 sec

Customer_Segment	Total_Customers	Total_Sales	Avg_RFM_Score
At Risk / Can't Lose	117892	2688786.75	1.50
Champions (Best Customers)	106195	2377571.70	4.50
Loyal Customers	97137	1977762.27	3.41
Lost / Hibernating	51306	820455.38	1.78
Recent New Customers	29074	413943.32	4.51

Explanation: Same join-and-aggregate pattern as section 6, adding COUNT(DISTINCT CustomerID) for the 'Total Customers' column. Sorting by Total_Sales DESC surfaces the 'Top Segment' KPI card automatically (the first row).

7.2 Top segment (single KPI card)

Objective: Reproduce the 'Top Segment' card, e.g. 'At Risk / Can't Lose'.

		13_top_segment.sql
1	SELECT cs.Customer_Segment, SUM(co.TotalAmount) AS Total_Sales	
2	FROM customer_segments cs	
3	JOIN clean_orders co ON co.CustomerID = cs.CustomerID	
4	GROUP BY cs.Customer_Segment	


```
5 ORDER BY Total_Sales DESC
6 LIMIT 1;
```

≡ Result Grid 1 row returned in 0.018 sec

Customer_Segment	Total_Sales
At Risk / Can't Lose	2688786.75

Explanation: ORDER BY Total_Sales DESC LIMIT 1 returns only the single highest-revenue segment — the value shown on the 'Top Segment' card.

8. Suggested Business Actions per Segment

Segment	Recommended Action
Champions (Best Customers)	Reward with loyalty perks / early access; protect this high-value group.
Loyal Customers	Upsell / cross-sell to push them toward Champion status.
Recent New Customers	Onboarding offers to build purchase frequency.
At Risk / Can't Lose	Win-back campaigns with personalised discounts before full churn.
Lost / Hibernating	Low-cost re-engagement emails; deprioritise heavy spend here.

9. Summary

This report reconstructed the full MySQL logic behind every KPI card and chart observed in the Power BI dashboard walkthrough (Customer Overview → Customer Segmentation → Recommendation), formatted as MySQL Workbench editor panes with syntax highlighting and Result Grid previews: total sales/orders/customers/AOV, monthly and country-wise revenue, RFM scoring with window functions, 5-way customer segmentation, and segment-level revenue/recommendation tables.

Run these statements in order (Section 2 → 8) inside MySQL Workbench against your real transactions table to reproduce the dashboard's exact figures (8.28M Total Sales, 401,604 Orders, 552 Customers, and the 5 RFM segments). Sample Result Grid values shown throughout are illustrative placeholders matching the video's on-screen numbers where visible.